UNIVERSIDADE FEDERAL DE SERGIPE

# Operadores bit-a-bit e de deslocamento

## Nove sins e nãos num único número

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

# O que você vai aprender hoje

- Um inteiro é uma fileira de bits: binário, octal e hexadecimal
- Os operadores `&`, `|`, `^` e `~`: um bit de cada vez
- Máscaras: ligar, desligar, inverter e testar bits
- Deslocamentos << e >>: montar máscaras e extrair campos
- A ordem dos bytes na memória: *little-endian* e *big-endian*
- Um conjunto inteiro dentro de um `unsigned`
- Exercícios

# A pergunta de hoje

# O problema: o que quer dizer 754?

### Até aqui

Um `int` foi sempre um número: uma nota, um tamanho, um índice. Vários sim/não pediam um campo para cada um, ou um vetor de `char`.

### O que vamos descobrir

Os 32 bits de um `unsigned` podem ser 32 sim/não independentes. C tem operadores para ler e mudar um bit de cada vez.

```
$ chmod 754 relatorio.sh
$ ls -l relatorio.sh
-rwxr-xr-- 1 ana alunos 812 nov 16 14:02 relatorio.sh
```

**A pergunta de hoje**

Como o número 754 guarda nove permissões — ler, escrever, executar para o dono, o grupo e os outros? E como mudar só uma delas?

Fonte: Kerrisk (2010), §15.4 [8]

# Permissões em UNIX: quem pode fazer o quê

## Três classes de usuários

Todo arquivo tem um dono e um grupo. As permissões valem separadamente para:

- `u`: o dono
- `g`: os membros do grupo
- `o`: todos os outros

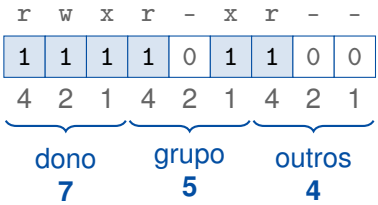
## Três permissões por classe

| | em arquivo | em diretório |
|---|---|---|
| `r` | ler o conteúdo | listar os nomes |
| `w` | alterar o conteúdo | criar e apagar entradas |
| `x` | executar como programa | entrar com `cd` |

Três classes $\times$ três permissões = **nove sim/não**. O kernel consulta só a primeira classe que casa: dono, senão grupo, senão outros.

Fonte: Kerrisk (2010), §15.4.1–15.4.3 [8]

# Lendo `rwxr-xr--` como 754




cada permissão é um bit:
`r = 4, w = 2, x = 1`

Cada classe ocupa **três bits**, e cada trio de bits é um dígito de 0 a 7: $4+2+1 = 7$, $4+1 = 5$, 4.

`chmod u+x` liga um bit; `chmod go-w` desliga dois. Ligar e desligar bits sem tocar nos vizinhos é o assunto de hoje.

Fonte: Kerrisk (2010), §15.4 [8]

# O programa de hoje: `permissoes.c` (1/2)

```c
#include <stdio.h>
#include <stdlib.h>
/* rwx do dono, do grupo, dos outros */
#define R 4u
#define W 2u
#define X 1u

/* imprime os nove bits: rwxr-xr-- */
void mostra(unsigned modo)
{
    const char *letras = "rwx";
    for (int i = 8; i >= 0; i--)
        putchar(modo & (1u << i)
                ? letras[(8 - i) % 3]
                : '-');
}
```

```c
/* aplica "u+x", "go-w", "a+r"... */
unsigned aplica(unsigned modo, const char *ordem)
{
    unsigned quem = 0, bits = 0, mascara = 0;
    const char *p = ordem;
    for (; *p == 'u' || *p == 'g' || *p == 'o'
           || *p == 'a'; p++)
        quem |= *p == 'u' ? 4u : *p == 'g' ? 2u
                : *p == 'o' ? 1u : 7u;
    char op = *p++;
    for (; *p != '\0'; p++)
        bits |= *p == 'r' ? R : *p == 'w' ? W
                : *p == 'x' ? X : 0u;
    if (quem & 4u) mascara |= bits << 6;
    if (quem & 2u) mascara |= bits << 3;
    if (quem & 1u) mascara |= bits;
    if (op == '+') modo |= mascara;
    else if (op == '-') modo &= ~mascara;
    return modo;
}
```

# permissoes.c (2/2): o main

```c
void mostra_linha(unsigned modo)
{
    printf("%03o ", modo);
    mostra(modo);
    putchar('\n');
}
```

%o imprime em **octal**, base 8; %03o completa com zeros até três dígitos. 0644 começa com zero: em C, isso também é octal.

O programa completo, em codigo/, aceita o modo e as ordens pela linha de comando: ./permissoes 754 u-x go+w.

```c
int main(void)
{
    unsigned modo = 0644;
    const char *ordens[] = {"u+x", "go+w",
                            "a-r", "o+rwx"};
    printf("%-6s", "");
    mostra_linha(modo);
    for (int i = 0; i < 4; i++) {
        modo = aplica(modo, ordens[i]);
        printf("%-6s", ordens[i]);
        mostra_linha(modo);
    }
    return 0;
}
```

# O que este programa imprime?

## Antes de compilar, escreva sua resposta

1. A primeira linha mostra o modo inicial `0644`. Que letras `mostra` imprime para ele?
2. Depois de `u+x`, o que muda — o número e as letras?
3. `go+w` dá escrita ao grupo e aos outros. Qual é o novo número?
4. `a-r` tira a leitura de todos. E `o+rwx`?
5. Repare em `modo |= mascara` e `modo &= ~mascara`. O que cada um faz com os bits que *não* estão na máscara?

# Compile, execute e compare

```
$ gcc -Wall -Wextra permissoes.c -o permissoes
$ ./permissoes
      644 rw-r--r--
u+x   744 rwxr--r--
go+w  766 rwxrw-rw-
a-r   322 -wx-w--w-
o+rwx 327 -wx-w-rwx
```

## O que o programa guarda

Um único `unsigned`. Cada dígito octal é uma classe: 6 = `rw-`, 4 = `r--`, 7 = `rwx`.

Cada ordem mexe só nos bits que pediu: `u+x` fez o 6 virar 7 sem tocar no grupo e nos outros; `a-r` subtraiu 4 de cada dígito que o tinha. Nenhuma conta de somar foi feita.

Fonte: Kerrisk (2010), §15.4 [8]

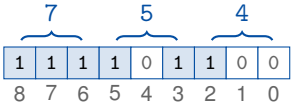
# Um inteiro é uma fileira de bits

# Definição: as bases 2, 8 e 16

## Posição vale potência da base

Na base 2, o dígito na posição $i$ (contando da direita, a partir de 0) vale $2^i$. Um `unsigned` de 32 bits guarda de 0 a $2^{32} - 1$.

| base | cada dígito | em C |
|---|---|---|
| 2 | 1 bit | — |
| 8 | 3 bits | `0754` |
| 16 | 4 bits | `0x1EC` |




octal: `0754`, três bits por dígito

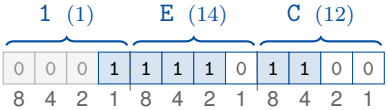
decimal: $256 + 128 + 64 + 32 + 8 + 4 = 492$

hexa: `1 1110 1100` = `0x1EC`

As permissões vêm em grupos de três: por isso o `chmod` usa octal.

Fonte: Bryant e O'Hallaron (2015), §2.1.1–2.1.3 [1]; ISO/IEC 9899:2018, §6.4.4.1 [5]

# Hexadecimal: quatro bits por dígito




1. agrupe de 4 em 4, *a partir da direita*
2. complete o grupo da esquerda com zeros
3. cada grupo vale de 0 a 15: troque pelo dígito

confere: $1 \cdot 16^2 + 14 \cdot 16 + 12 = 256 + 224 + 12 = 492$

Um byte são dois dígitos hexa; um `unsigned` de 32 bits, oito: `0x000001EC`.

| hexa | bits | dec | hexa | bits | dec |
|---|---|---|---|---|---|
| 0 | 0000 | 0 | 8 | 1000 | 8 |
| 1 | 0001 | 1 | 9 | 1001 | 9 |
| 2 | 0010 | 2 | A | 1010 | 10 |
| 3 | 0011 | 3 | B | 1011 | 11 |
| 4 | 0100 | 4 | C | 1100 | 12 |
| 5 | 0101 | 5 | D | 1101 | 13 |
| 6 | 0110 | 6 | E | 1110 | 14 |
| 7 | 0111 | 7 | F | 1111 | 15 |

```c
/* 0x1EC e 0x1ec: mesmo valor */
unsigned m = 0x1EC;
printf("%x %#X\n", m, m);
/* saida: 1ec 0X1EC */
```

Fonte: Bryant e O'Hallaron (2015), §2.1.1 [1]; ISO/IEC 9899:2018, §6.4.4.1 e §7.21.6.1 [5]

# E se esquecer o zero?

```c
/* decimal.c */
int main(void)
{
    unsigned modo = 754; /* rwxr-xr-- ? */
    printf("%o ", modo);
    mostra(modo);
    putchar('\n');
    return 0;
}
```

A mesma `mostra` de `permissoes.c`. Só falta um caractere.

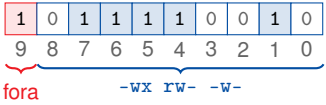
**Pense antes de passar o slide**

1. O `gcc -Wall` reclama de alguma coisa?
2. O que o `printf("%o")` imprime?
3. E as nove letras?

# Execute e compare

```
$ gcc -Wall -Wextra decimal.c -o decimal
$ ./decimal
1362 -wxrw--w-
```

$754 = 512 + 128 + 64 + 32 + 16 + 2$




Nenhum aviso: `754` é um número decimal perfeitamente válido. Em binário ele ocupa dez bits; em octal, `1362`. O bit 9 nem é olhado, e os nove de baixo dão permissões que ninguém pediu. Com bits, escreva as constantes na base que mostra os bits: octal (`0754`) ou hexadecimal (`0x1EC`).

Fonte: ISO/IEC 9899:2018, §6.4.4.1 [5]; Kernighan e Ritchie (1988), §2.3 [6]

# Por que `unsigned`?

## Em `int`, o bit 31 é o sinal

Num `int` de 32 bits, o bit mais alto vale $-2^{31}$ (complemento de dois). Ligar o bit 31 faz o número ficar negativo; deslocar um negativo para a direita depende do compilador.

## Em `unsigned`, todo bit é só um bit

Os 32 bits valem $2^0$ a $2^{31}$, sem sinal, e as contas são módulo $2^{32}$. Os resultados de `&`, `|`, `^`, `~`, `<<` e `>>` são os mesmos em qualquer compilador.

Regra prática: quando o número é uma coleção de bits, e não uma quantidade, use `unsigned` — e escreva as constantes com `u`: `1u`, `4u`, `0x80000000u`.

Fonte: Bryant e O'Hallaron (2015), §2.2 [1]; King (2008), §20.1 [9]

# Recapitulando: um inteiro é uma fileira de bits

## O que aprendemos

- o bit $i$ vale $2^i$; um `unsigned` de 32 bits são 32 sim/não;
- cada dígito octal são 3 bits, cada dígito hexa são 4: por isso `chmod` fala octal;
- em C, `0754` é octal, `0x1EC` é hexa e `754` é decimal — e o compilador não avisa;
- para mexer em bits, use `unsigned`.

# Os operadores &, |, ^ e ~

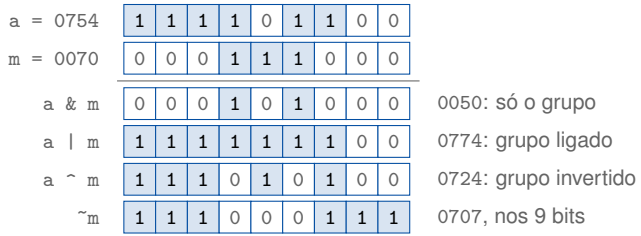
# Definição: operadores bit a bit

## Um bit de cada vez

Cada bit do resultado depende só dos bits na **mesma posição** dos operandos.

| *a* | *b* | a&b | a\|b | a^b | ~a |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 |

e, ou, ou-exclusivo, não.




Fonte: Kernighan e Ritchie (1988), §2.9 [6]; Bryant e O'Hallaron (2015), §2.1.7 [1]

# `&` não é `&&`

| expressão | valor | por quê |
|---|---|---|
| `6 & 1` | 0 | `110` e `001`: nada em comum |
| `6 && 1` | 1 | os dois são $\neq 0$ |
| `6 \| 1` | 7 | `111` |
| `6 \|\| 1` | 1 | verdadeiro |
| `~6` | `-7` | os 32 bits invertidos |
| `!6` | 0 | não verdadeiro é falso |

**Lógicos: o número todo é sim ou não**

`&&`, `||` e `!` olham o número inteiro e dão 0 ou 1. `&&` e `||` param assim que sabem a resposta.

**Bit a bit: 32 contas em paralelo**

`&`, `|`, `^` e `~` dão um número inteiro e sempre avaliam os dois lados.

`if (modo && W)` é verdadeiro para qualquer modo diferente de zero — e compila sem aviso.

Fonte: Kernighan e Ritchie (1988), §2.9 [6]; Bryant e O'Hallaron (2015), §2.1.8 [1]

# O ~ inverte os 32 bits, não só os nove

```c
unsigned m = 0444;      /* r para todos */
printf("%o\n", ~m);     /* 37777777333 */
printf("%o\n", ~m & 0777);      /* 333 */
printf("%o\n", 0766 & ~m);      /* 322 */
```

`~m` liga os 23 bits de cima, que estavam zerados. Para ver só as permissões, corte com `& 0777`.

Em `modo &= ~mascara` isso não atrapalha: os bits altos de `modo` já eram zero, e `0 & 1` continua 0.

O tamanho conta: `~` é o único dos quatro que transforma zeros em uns. Antes de imprimir ou comparar o resultado, pergunte quantos bits importam.

Fonte: ISO/IEC 9899:2018, §6.5.3.3 [5]; King (2008), §20.1 [9]

# Recapitulando: `&`, `|`, `^` e `~`

## O que aprendemos

- os quatro operadores trabalham posição a posição, nos 32 bits ao mesmo tempo;
- `&` só deixa passar onde os dois têm 1; `|` liga onde qualquer um tem 1; `^` marca onde são diferentes;
- `&&`, `||` e `!` são outra coisa: olham o número inteiro e devolvem 0 ou 1;
- `~` também inverte os bits que você não está usando.

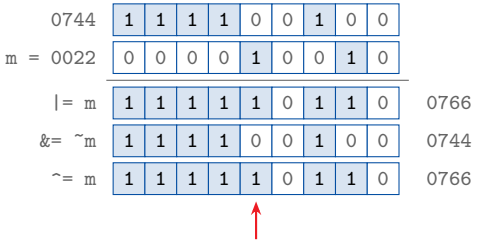
# Máscaras

# Definição: máscara

**Máscara**

Um número com 1 exatamente nos bits em que se quer mexer. Combinada com o operador certo, muda esses bits e deixa os outros como estavam.

| quero | escrevo |
|---|---|
| ligar | `x \|= m` |
| desligar | `x &= ~m` |
| inverter | `x ^= m` |
| testar | `(x & m) != 0` |

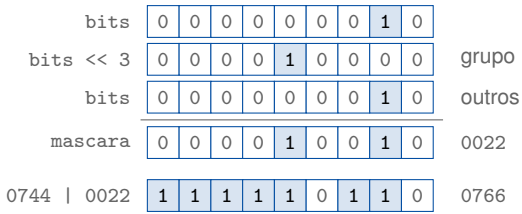



Fonte: Kernighan e Ritchie (1988), §2.9 [6]; Deitel e Deitel (2011), cap. 10 [3]

# Como `aplica` monta a máscara de `go+w`

```
/* quem = 2|1, bits = W = 2 */
if (quem & 4u) mascara |= bits << 6;
if (quem & 2u) mascara |= bits << 3;
if (quem & 1u) mascara |= bits;
modo |= mascara;
```

`quem` também é uma coleção de bits: 4 é o dono, 2 o grupo, 1 os outros, e `a` liga os três (7).




Fonte: Kerrisk (2010), §15.4 [8]; código da aula

# E se testar sem parênteses?

```c
/* precedencia.c: 1 se a classe (6 = dono,
   3 = grupo, 0 = outros) NAO pode escrever */
int nao_escreve(unsigned modo, int classe)
{
    if (modo & W << classe == 0)
        return 1;
    return 0;
}
/* main: para 0644, 0664 e 0444, imprime
   se o dono e o grupo escrevem */
```

**Pense antes de passar o slide**

1. Em `0644`, o dono escreve? E o grupo?
2. Em `0444`, alguém escreve?
3. O `gcc -Wall` diz alguma coisa?

# Compile, execute e compare

```
$ gcc -Wall -Wextra precedencia.c -o precedencia
precedencia.c:9:28: warning: suggest parentheses around comparison in operand
  of '&' [-Wparentheses]
    9 |     if (modo & W << classe == 0)
$ ./precedencia
644: dono escreve, grupo escreve
664: dono escreve, grupo escreve
444: dono escreve, grupo escreve
```

Todo mundo escreve — até em `0444`, que é só leitura. O `gcc` avisou, mas é só um aviso: o executável foi gerado.

# A resposta: `&` perde para `==`

| da mais forte para a mais fraca | |
|---|---|
| `~ !` | unários |
| `* / %` | |
| `+ -` | |
| `<< >>` | deslocamentos |
| `< <= > >=` | |
| `== !=` | |
| `&`, depois `^`, depois `\|` | bit a bit |
| `&&`, depois `\|\|` | lógicos |

A expressão foi lida assim:

```
modo & ((W << classe) == 0)
```

`W << classe` nunca é zero, então a comparação dá 0, e `modo & 0` é sempre 0: a função nunca devolve 1.

A correção são parênteses em volta do `&`:

```
if ((modo & W << classe) == 0)
```

Kernighan e Ritchie avisam: como `&`, `^` e `|` ficam abaixo de `==`, todo teste de bits precisa de parênteses.

Fonte: Kernighan e Ritchie (1988), §2.9 e §2.12, tabela 2-1 [6]

# Flags que você vai usar na próxima aula

```c
#include <fcntl.h>
/* cria se nao existe, so escrita,
   apaga o conteudo antigo */
int fd = open("saida.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);
```

```c
enum {
    NEGRITO    = 1u << 0,
    ITALICO    = 1u << 1,
    SUBLINHADO = 1u << 2,
};
unsigned estilo = NEGRITO | SUBLINHADO;
```

Cada opção é uma potência de 2 — um bit só. Juntar opções é `|`; perguntar por uma é `&`.

O `open` recebe as opções num `int` e as permissões do arquivo novo em outro: `0644`, em octal, como no `chmod`.

Com `enum` (aula de enumerações), as constantes ganham nome e tipo.

Fonte: Kerrisk (2010), §4.3, `open()` [8]; King (2008), §20.1 [9]

# Recapitulando: máscaras

## O que aprendemos

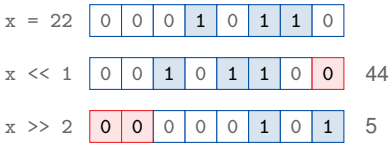
- máscara: 1 nos bits em que se quer mexer, 0 nos outros;
- ligar é `|=`, desligar é `&= ~`, inverter é `^=`, testar é `&`;
- `&`, `^` e `|` têm precedência *menor* que `==`: ponha parênteses sempre;
- flags são potências de 2 combinadas com `|`.

# Deslocamentos

# Definição: `x << k` e `x >> k`

## Deslocamento

`x << k` empurra os bits de `x` $k$ casas para a esquerda; entram zeros à direita, e os que passam da borda se perdem. `x >> k` empurra para a direita.




em vermelho, os zeros que entraram

Em `unsigned`, sem estouro, `x << k` $= x \cdot 2^k$ e `x >> k` $= \lfloor x/2^k \rfloor$.

Com << se **fabrica** uma máscara: `1u << k` tem um único 1, no bit $k$. Com >> se **traz** um campo para a direita, onde é fácil olhar.

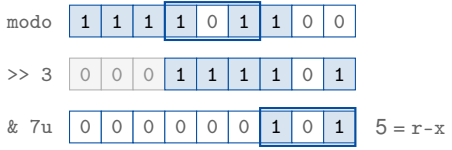
Fonte: Kernighan e Ritchie (1988), §2.9 [6]; Bryant e O'Hallaron (2015), §2.1.9 [1]

# Extraindo um campo: as permissões do grupo

```c
unsigned modo = 0754;
unsigned grupo = (modo >> 3) & 7u;  /* 5 */
unsigned bit_i = (modo >> i) & 1u;  /* 0 ou 1
     */
```

Primeiro desloca, para o campo chegar ao bit 0; depois corta com uma máscara da largura do campo: `7u` são três bits.

Na direção contrária, *colocar* um campo é `(valor & 7u) << 3`.




Fonte: Bryant e O'Hallaron (2015), §2.1.9 [1]; King (2008), §20.1 [9]

# E se o bit for o 31?

```c
/* mascaras.c */
int main(void)
{
    for (int k = 28; k <= 32; k++) {
        int m = 1 << k;
        printf("k = %2d: %11d = 0x%08x\n",
               k, m, (unsigned) m);
    }
    return 0;
}
```

A máscara do bit $k$, para os bits mais altos de um `int` de 32 bits — e um além.

**Pense antes de passar o slide**

1. Para $k = 28, \ldots, 30$, o que sai em decimal e em hexa?
2. E para $k = 31$? O número é positivo?
3. O bit 32 não existe. O que você espera de `1 << 32`?

# Execute e compare

```
$ gcc -Wall -Wextra mascaras.c -o mascaras && ./mascaras
k = 28:   268435456 = 0x10000000
k = 29:   536870912 = 0x20000000
k = 30:  1073741824 = 0x40000000
k = 31: -2147483648 = 0x80000000
k = 32:           1 = 0x00000001
$ gcc -g -fsanitize=undefined mascaras.c -o mascaras-ubsan && ./mascaras-ubsan
mascaras.c:7:19: runtime error: left shift of 1 by 31 places cannot be
  represented in type 'int'
```

Sem aviso na compilação. Em $k = 31$, o 1 caiu no bit de sinal; em $k = 32$, voltou para o bit 0. O *sanitizer* de comportamento indefinido (`-fsanitize=undefined`), primo do `-fsanitize=address`, aponta a linha.

# A resposta: dois comportamentos indefinidos

### `1 << 31` em `int`

$2^{31}$ não cabe num `int`: deslocar um 1 para dentro do bit de sinal é indefinido. A correção é `1u`, que é `unsigned`.

### `x << 32` num tipo de 32 bits

Deslocar por $k \geq$ largura (ou $k < 0$) é indefinido, mesmo em `unsigned`. O processador costuma usar $k \bmod 32$ — daí o 1 de volta —, mas nada garante.

Trocando para `unsigned m = 1u << k;`, o sanitizer muda de queixa:

```
mascaras.c:7:25: runtime error: shift
  exponent 32 is too large for 32-bit
  type 'unsigned int'
```

### Regras

- desloque `unsigned`: `1u << k`;
- garanta $0 \leq k < 32$;
- `>>` de um negativo depende do compilador.

Fonte: ISO/IEC 9899:2018, §6.5.7 [5]; Bryant e O'Hallaron (2015), §2.1.9 [1]

# Recapitulando: deslocamentos

## O que aprendemos

- << multiplica por $2^k$ e fabrica máscaras: `1u << k`;
- >> divide por $2^k$ e traz campos para baixo: `(x >> s) & m`;
- `1 << 31` em `int` e qualquer deslocamento por $k \geq 32$ são indefinidos;
- `-fsanitize=undefined` encontra esses casos na execução.

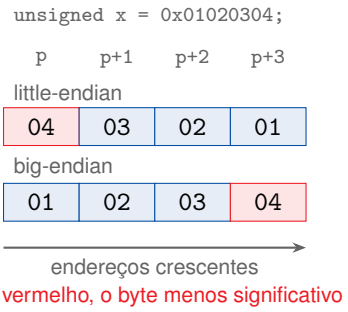
# A ordem dos bytes na memória

# Definição: *little-endian* e *big-endian*

## Ordem dos bytes (*endianness*)

Um `unsigned` de 32 bits ocupa 4 bytes, em 4 endereços seguidos. **Little-endian**: o byte menos significativo fica no menor endereço. **Big-endian**: o mais significativo fica no menor endereço.

x86 e ARM (no modo usual) são little-endian; os protocolos de rede usam big-endian. Os nomes vêm das *Viagens de Gulliver*: a guerra entre quem quebra o ovo pela ponta menor e pela maior.




Fonte: Bryant e O'Hallaron (2015), §2.1.3 [1]; Cohen (1981) [2]

# E se olhar os bytes um a um?

```c
/* bytes.c */
int main(void)
{
    unsigned x = 0x01020304;
    unsigned char *p = (unsigned char *) &x;
    for (int i = 0; i < 4; i++)
        printf("p[%d] = %02x\n", i, p[i]);
    printf("(x >> 24) & 0xFF = %02x\n",
           (x >> 24) & 0xFFu);
    return 0;
}
```

`p` aponta para o primeiro byte de `x`; `p[i]` é o byte no endereço `p + i`.

**Pense antes de passar o slide**

1. `p[0]` é 01 ou 04?
2. E `(x >> 24) & 0xFF`?
3. Num processador big-endian, qual das duas respostas muda?

# Execute e compare

```
$ gcc -Wall -Wextra bytes.c -o bytes && ./bytes
p[0] = 04
p[1] = 03
p[2] = 02
p[3] = 01
(x >> 24) & 0xFF = 01
```

A memória guarda os bytes “ao contrário”: este computador (ARM, como os PCs x86) é little-endian. Num big-endian, `p[0]` seria `01`. Já `(x >> 24) & 0xFF` dá `01` em qualquer máquina: >> vale $\lfloor x/2^{24} \rfloor$, uma conta sobre o *valor*, que não sabe como os bytes estão arrumados.

Fonte: Bryant e O’Hallaron (2015), §2.1.3 [1]; ISO/IEC 9899:2018, §6.2.6.1 [5]

# A resposta: >> olha o valor; o ponteiro, a memória

```c
/* grava x em big-endian, em qualquer maquina */
void grava_be(unsigned char b[4], unsigned x)
{
    b[0] = (x >> 24) & 0xFFu;
    b[1] = (x >> 16) & 0xFFu;
    b[2] = (x >> 8) & 0xFFu;
    b[3] = x & 0xFFu;
}
unsigned le_be(const unsigned char b[4])
{
    return (unsigned) b[0] << 24
         | (unsigned) b[1] << 16
         | (unsigned) b[2] << 8 | b[3];
}
```

## Quando a ordem importa

Só quando os bytes saem do programa: um `int` gravado com `fwrite` num arquivo binário (próxima aula) ou enviado pela rede. Quem lê, em outra máquina, pode ler na outra ordem.

A saída portátil é escolher uma ordem para o arquivo e montar os bytes com >> e <<, como ao lado. Para a rede, `<arpa/inet.h>` tem `htonl` e `ntohl`.

Fonte: Kerrisk (2010), §59.2 [8]; Bryant e O'Hallaron (2015), §2.1.3 [1]

# Recapitulando: a ordem dos bytes na memória

## O que aprendemos

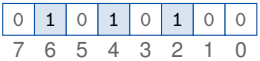
- little-endian guarda o byte menos significativo no menor endereço; big-endian, o mais significativo;
- x86 e ARM são little-endian; a rede é big-endian;
- >>, << e `&` trabalham no valor e dão o mesmo resultado em qualquer máquina;
- ler um `int` byte a byte por ponteiro, ou gravá-lo com `fwrite`, expõe a ordem da máquina.

# Um conjunto num `unsigned`

# O elemento *i* está no conjunto se o bit *i* vale 1

```c
typedef unsigned Conjunto;   /* 0 a 31 */

Conjunto insere(Conjunto c, int i)
{ return c | 1u << i; }
Conjunto retira(Conjunto c, int i)
{ return c & ~(1u << i); }
int pertence(Conjunto c, int i)
{ return (c >> i) & 1u; }
```




$\{2, 4, 6\}$

| conjuntos | bits |
|---|---|
| $A \cup B$ | `a \| b` |
| $A \cap B$ | `a & b` |
| $A \setminus B$ | `a & ~b` |
| $A \triangle B$ | `a ^ b` |
| $A = \emptyset$? | `a == 0` |

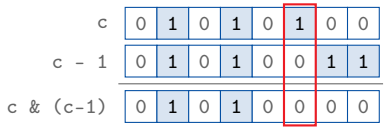
Uma união de 32 elementos é **uma** instrução, sem laço.

Fonte: Knuth (2011), §7.1.3 [10]; Kernighan e Ritchie (1988), §2.9 [6]

# Quantos elementos? Apague o 1 mais à direita

```c
int conta(Conjunto c)
{
    int n = 0;
    while (c != 0) {
        c &= c - 1;
        n++;
    }
    return n;
}
```

O laço dá uma volta por elemento, não por bit.




Subtrair 1 transforma o 1 mais à direita em 0 e os zeros à direita dele em 1; o `&` apaga tudo isso de uma vez. O resto não muda.

Fonte: Kernighan e Ritchie (1988), §2.10, exercício 2-9 [6]; Warren (2013), §2-1 [11]

# conjunto.c: pares e múltiplos de 3 até 20

```
$ gcc -Wall -Wextra conjunto.c -o conjunto && ./conjunto
pares      { 2 4 6 8 10 12 14 16 18 20 }  10 elementos
mult3      { 3 6 9 12 15 18 }  6 elementos
p | m      { 2 3 4 6 8 9 10 12 14 15 16 18 20 }  13 elementos
p & m      { 6 12 18 }  3 elementos
p & ~m     { 2 4 8 10 14 16 20 }  7 elementos
p ^ m      { 2 3 4 8 9 10 14 15 16 20 }  10 elementos
sem o 6    { 2 4 8 10 12 14 16 18 20 }  9 elementos
```

`p & m` são os múltiplos de 6; `p ^ m`, os que estão em exatamente um dos dois. Para mais de 32 elementos, use um vetor de `unsigned`: o elemento *i* é o bit `i % 32` da palavra `i / 32`.

Fonte: Knuth (2011), §7.1.3 [10]

# Um bit por número: o crivo de Eratóstenes

```c
/* o bit i % 32 da palavra v[i / 32] vale 1 se i nao e primo */
#define LIGA(v, i)  ((v)[(i) >> 5] |= 1u << ((i) & 31))
#define TESTA(v, i) (((v)[(i) >> 5] >> ((i) & 31)) & 1u)
```

`i >> 5` é `i / 32` e `i & 31` é `i % 32`: com potências de 2, dividir e tirar o resto viram deslocamento e máscara.

```
$ ./crivo
char: 5761455 primos, 100.0 MB, 0.62 s
bits: 5761455 primos,  12.5 MB, 0.72 s
```

Primos até $10^8$: o mesmo resultado com um oitavo da memória e quase o mesmo tempo (medido num notebook; no seu computador os números mudam).

Fonte: Knuth (2011), §7.1.3 [10]; Bryant e O'Hallaron (2015), §2.3.7 [1]

# Recapitulando: um conjunto num `unsigned`

## O que aprendemos

- o bit *i* diz se *i* está no conjunto: 32 elementos numa palavra;
- união, interseção, diferença e diferença simétrica são `|`, `&`, `& ~` e `^`;
- `c &= c - 1` apaga o 1 mais à direita: contar custa uma volta por elemento;
- com um vetor de `unsigned`, um bit por número: o crivo cabe em um oitavo da memória.

# Mãos à obra

# Altere o programa

## Tarefa 1 — a ordem `=`

`u=rx` deixa o dono com exatamente `r-x`, qualquer que fosse o antes, sem mexer nas outras classes. Implemente em `aplica`: primeiro desligue as três permissões das classes de `quem`, depois ligue as pedidas.

## Tarefa 2 — a ordem `^`

Uma ordem que não existe no `chmod`: `go^w` inverte a escrita do grupo e dos outros. Que operador resolve em uma linha? Aplicar duas vezes volta ao modo original?

## Tarefa 3 — `mostra` sem `1u << i`

Reescreva `mostra` deslocando o *modo*: teste sempre o bit 8 com `modo & 0400` e faça `modo <<= 1` a cada volta.

# Exercícios (1/3)

## Exercício 1 — os bits de um número

Escreva `void mostra_bits(unsigned x, int n)`, que imprime os *n* bits mais baixos de `x`, do mais alto para o mais baixo, com um espaço a cada 4. Use-a para conferir `0754`, `~0u` e `-1`.

## Exercício 2 — distância de Hamming

Em quantos bits `a` e `b` diferem? Use `^` e `conta`. Quantos bits mudam de um número para o seguinte, de 0 a 15?

## Exercício 3 — de letras para número

Escreva `unsigned de_texto(const char *s)`, o inverso de `mostra`: `"rwxr-x---"` vira `0750`.

Fonte: Kernighan e Ritchie (1988), §2.9 [6]; Warren (2013), cap. 5 [11]

# Exercícios (2/3)

## Exercício 4 — cores `0xRRGGBB`

Uma cor de 24 bits guarda vermelho, verde e azul, um byte cada: `rgb(r, g, b)` empacota; `vermelho(c)`, `verde(c)`, `azul(c)`, que desempacotam. Depois `cinza(c)`: a média dos três, repetida nos três bytes.

## Exercício 5 — potências de 2

Por que `x != 0 && (x & (x - 1)) == 0` diz se `x` é potência de 2? Escreva `proxima_potencia(x)`: a menor $\geq x$.

Fonte: Warren (2013), §2-1 e §3-2 [11]; Bryant e O'Hallaron (2015), §2.1 [1]

# Exercícios (3/3)

## Exercício 6 — rotação

`rot_esq(x, k)` gira os 32 bits: o que sai pela esquerda volta pela direita. Cuidado com $k = 0$.

## Exercício 7 — trocando a ordem dos bytes

`troca_bytes(x)` inverte os 4 bytes: `0x01020304` vira `0x04030201`. Depois `eh_little()`: 1 se a máquina é little-endian.

Fonte: Warren (2013), §3-2 e §7-1 [11]; Bryant e O'Hallaron (2015), §2.1.3 [1]

# Desafio

## Duas pilhas de mesmo peso

Leia $n \leq 20$ pesos e divida-os em duas pilhas com a menor diferença de peso possível. Imprima a diferença e os pesos de uma das pilhas.

- cada subconjunto é um número de 0 a $2^n - 1$: o bit *i* diz se o peso *i* vai para a primeira pilha;
- percorra todos com `for (mask = 0; mask < 1u << n; mask++);`
- para cada um, some os pesos dos bits ligados.

```
$ echo 5 3 1 4 2 2 | ./pilhas
diferenca: 0
pilha: 3 1 2
```

Quantos subconjuntos há para $n = 20$? E para $n = 40$? Por que ninguém conhece um método muito mais rápido no pior caso?

Fonte: Knuth (2011), §7.2.1.1 [10]; Garey e Johnson (1979), SP12 [4]

# Fechamento

## O que mudou hoje

Um inteiro deixou de ser só uma quantidade: virou uma fileira de 32 sim/não, que se lê e se muda com máscaras e deslocamentos.

## O que ficou

Arquivos binários: bytes de 8 bits, numa ordem que depende da máquina, abertos com flags e permissões como `0644`.

## Para levar

- ligar `|=`, desligar `&= ~`, inverter `^=`, testar `&`;
- `1u << k` fabrica a máscara; `(x >> s) & m` extrai o campo;
- use `unsigned`, constantes em octal ou hexa e parênteses em volta de `&`;
- deslocar por $k \geq 32$ ou `1 << 31` em `int`: indefinido.

Próxima aula: arquivos texto e binários.

# Referências I



Randal E. Bryant and David R. O'Hallaron.
*Computer Systems: A Programmer's Perspective*.
Pearson, Boston, 3 edition, 2015.
Seções 2.1.1–2.1.9: hexadecimal, operações bit a bit, lógicas e de deslocamento.


Danny Cohen.
On holy wars and a plea for peace.
*Computer*, 14(10):48–54, 1981.
Origem dos termos *big-endian* e *little-endian* em computação.

# Referências II



Paul Deitel and Harvey Deitel.
*C: como programar*.
Pearson Prentice Hall, São Paulo, 6 edition, 2011.
Capítulo 10: manipulação de bits.


Michael R. Garey and David S. Johnson.
*Computers and Intractability: A Guide to the Theory of NP-Completeness*.
W. H. Freeman, San Francisco, 1979.
Problema SP12: partição.

# Referências III


ISO/IEC.
Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seções 6.4.4.1 (constantes inteiras), 6.5.3.3 (~), 6.5.7 (deslocamentos) e 6.5.10–6.5.12 (&, ^, |).

Brian W. Kernighan and Dennis M. Ritchie.
*The C Programming Language*.
Prentice Hall, Englewood Cliffs, NJ, 2 edition, 1988.
Seções 2.9 (operadores bit a bit), 2.10 (atribuição composta, exercício 2-9) e 2.12 (precedência).

# Referências IV



Brian W. Kernighan and Dennis M. Ritchie.
*C: a linguagem de programação — padrão ANSI*.
Campus, Rio de Janeiro, 1989.


Michael Kerrisk.
*The Linux Programming Interface*.
No Starch Press, San Francisco, 2010.
Seção 15.4: permissões de arquivos.


K. N. King.
*C Programming: A Modern Approach*.
W. W. Norton, New York, 2 edition, 2008.
Capítulo 20: programação de baixo nível.

# Referências V



Donald E. Knuth.
*The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*.
Addison-Wesley, Upper Saddle River, NJ, 2011.
Seções 7.1.3 (truques bit a bit) e 7.2.1.1 (gerando todas as $n$-uplas).


Henry S. Warren.
*Hacker's Delight*.
Addison-Wesley, Upper Saddle River, NJ, 2 edition, 2013.
Seção 2-1: manipulando os bits mais à direita; capítulo 5: contando bits.